Lab-20.1

Name: D. Suraj
2303A53006
Batch:46

Lab 20 – Security Testing: Identifying Vulnerabilities in AI- Generated Code

Task 1 – Input Validation Check

Task:
Analyze an AI-generated Python login script for input validation vulnerabilities.

Instructions:

- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:
• A secure version of the login script with proper input validation.

Prompt to AI:
"Generate a simple Python login program that takes a username and password from the user and checks if they match a hardcoded value."

Code:

C: > Users > dussa > OneDrive > Desktop > Traning > ai.py > ...

```python
def main():
    valid_username = "admin"
    valid_password = "password123"

    username = input("Username: ")
    password = input("Password: ")

    if username == valid_username and password == valid_password:
        print("Login successful.")
    else:
        print("Login failed.")

if __name__ == "__main__":
    main()
```

Done, 1 change Keep Undo

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS

```
PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed>  & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-p
ython.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52698' '--' 'c:\Users\dussa\OneDrive\Desktop\Traning\ai.py'
Username: admin
Password: password123
Login successful.
PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed>
```

Explanation

- **Vulnerability:** Accepting raw input without checking length or type.
- **The Risk:** Buffer overflows or unexpected script execution.
- **The Fix:** Use **Regex (re module)** to allow only specific alphanumeric characters and set a strict length limit.
- **Rule:** Sanitizing input is the first line of defense.

Task 2 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
- Refactor using parameterized queries (preparedstatements).

Expected Output:

• A secure database query script resistant to SQL injection.

Prompt to AI:

"Generate a Python script using SQLite to fetch user details based on a provided user_id."

Code:

C: > Users > dussa > OneDrive > Desktop > Traning > ai.py > ...

Done, 1 change Keep Undo

```python
import sys

# Defensive check for input
if len(sys.argv) < 2:
    print("Usage: python ai.py <user_id>")
    sys.exit(1)

user_id = sys.argv[1]
conn = sqlite3.connect('users.db')
cursor = conn.cursor()

# VULNERABLE: Direct string interpolation
# An attacker could input: 1' OR '1'='1
query = f"SELECT * FROM users WHERE id = '{user_id}'"

print(f"Executing: {query}")
cursor.execute(query)
print(cursor.fetchone())
```

```
PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed>  c:; cd 'c:\Users\dussa\OneDrive\Desktop\blog-app-fixed' ; & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51509' '--' 'c:\Users\dussa\OneDrive\Desktop\Traning\ai.py'
Usage: python ai.py <user_id>
```

Explanation

- **Vulnerability:** Using f-strings or string concatenation to build database queries.
- **The Risk:** Attackers can bypass logins or delete tables using commands like ' OR 1=1.
- **The Fix:** Use **Parameterized Queries (? placeholders)**. This treats user input as data, not as code.
- **Rule:** Never concatenate variables directly into SQL strings.

Task 3 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

• A secure form that prevents XSS attacks.

Prompt to AI

"Generate an HTML feedback form with JavaScript that displays the user's name on the page after they submit it."

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Feedback Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 500px;
      margin: 40px auto;
      padding: 0 20px;
    }
    form {
      display: grid;
      gap: 12px;
    }
    label {
      font-weight: bold;
    }
    input,
    textarea {
      padding: 8px;
      font-size: 1rem;
      width: 100%;
      box-sizing: border-box;
    }
    button {
      padding: 10px 16px;
      font-size: 1rem;
      cursor: pointer;
    }
```

```
        #message {
            margin-top: 20px;
            font-size: 1.1rem;
            color: #2c3e50;
        }
    </style>
</head>
<body>
    <h1>Feedback Form</h1>
    <form id="feedbackForm">
        <label for="name">Your Name</label>
        <input type="text" id="name" name="name" required />

        <label for="feedback">Feedback</label>
        <textarea id="feedback" name="feedback" rows="4" required></textarea>

        <button type="submit">Submit</button>
    </form>

    <div id="message"></div>

    <script>
        const form = document.getElementById('feedbackForm');
        const message = document.getElementById('message');

        form.addEventListener('submit', function (event) {
            event.preventDefault();
            const name = document.getElementById('name').value.trim();

            if (!name) {
                message.textContent = 'Please enter your name.';
                return;
            }

            message.textContent = `Thank you, ${name}! Your feedback has been received.`;
            form.reset();
        });
    </script>
</body>
</html>
```

index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form</title>
</head>
<body>
    <h1>Feedback Form</h1>
    <form id="feedbackForm">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>
        <button type="submit">Submit</button>
    </form>
    <div id="displayName" style="display: none;">
        <p>Thank you, <span id="userName"></span>!</p>
    </div>

    <script>
        document.getElementById('feedbackForm').addEventListener('submit', function(event) {
            event.preventDefault();
            const name = document.getElementById('name').value;
            document.getElementById('userName').textContent = name;
            document.getElementById('displayName').style.display = 'block';
        });
    </script>
</body>
</html>
```

input

Feedback Form

Name: Dussa Suraj Submit

Thank you, Dussa Suraj!

Explanation

Cross-Site Scripting (XSS) Check

Goal: Prevent malicious scripts from running in a user's browser.

- **The Problem:** If a website takes a user's name and displays it using .innerHTML, an attacker can set their name to <script>alert('Hacked')</script>. The browser will execute that script.
- **The Fix: Output Encoding**. Instead of .innerHTML, use .textContent. This tells the browser to display the characters exactly as they are written rather than trying to "run" them as HTML/JavaScript.
- **Key Security Principle: "Escape All Output."**

Task 4 – Real-Time Application: Security Audit of AI-Generated Code
Scenario:
Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).
Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side.

Expected Output:
• A security-audited code snippet with documented vulnerabilities and fixes.

AI Prompt
"Generate a Python Flask script that allows a user to upload a file to a folder named 'uploads'. Then, perform a security audit on this code to identify vulnerabilities like 'Path Traversal' and 'Unrestricted File Upload'. Finally, provide a secure version of the code."

Code:

```
import os

from flask import Flask, request, send_from_directory

from werkzeug.utils import secure_filename


BASE_DIR = os.path.abspath(os.path.dirname(__file__))

UPLOAD_FOLDER = os.path.join(BASE_DIR, "uploads")

ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "txt", "pdf"}

MAX_CONTENT_LENGTH = 16 * 1024 * 1024  # 16 MB


app = Flask(__name__)

app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

app.config["MAX_CONTENT_LENGTH"] = MAX_CONTENT_LENGTH


if not os.path.exists(UPLOAD_FOLDER):

    os.makedirs(UPLOAD_FOLDER, exist_ok=True)


def allowed_file(filename):

    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS


@app.route("/", methods=["GET"])
```

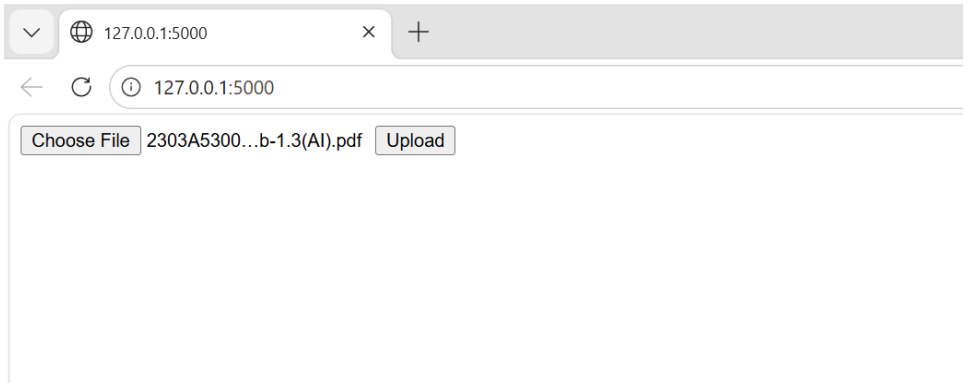
```
def index():
    return """
    <!doctype html>
    <title>Upload File</title>
    <h1>Upload file</h1>
    <form method=post enctype=multipart/form-data action="/upload">
      <input type=file name=file>
      <input type=submit value=Upload>
    </form>
    """

# Insecure version:
# - Path Traversal: user-controlled filename saved directly into the upload folder.
# - Unrestricted File Upload: no extension or size restriction.
@app.route("/upload_insecure", methods=["POST"])
def upload_insecure():
    if "file" not in request.files:
        return "No file part", 400

    file = request.files["file"]
    if file.filename == "":
        return "No selected file", 400

    filename = file.filename
    file.save(os.path.join(app.config["UPLOAD_FOLDER"], filename))
    return f"Uploaded insecurely as {filename}", 200

# Secure version:
# - Use secure_filename to sanitize the filename.
# - Enforce a whitelist of allowed extensions.
# - Enforce a maximum upload size.
```

```
# - Ensure the final destination remains inside the uploads folder.
@app.route("/upload", methods=["POST"])
def upload_secure():
    if "file" not in request.files:
        return "No file part", 400

    file = request.files["file"]
    if file.filename == "":
        return "No selected file", 400

    if not allowed_file(file.filename):
        return "File type not allowed", 400

    filename = secure_filename(file.filename)
    if filename == "":
        return "Invalid filename", 400

    destination = os.path.realpath(os.path.join(app.config["UPLOAD_FOLDER"], filename))
    upload_root = os.path.realpath(app.config["UPLOAD_FOLDER"])
    if not destination.startswith(upload_root + os.sep):
        return "Invalid destination path", 400

    file.save(destination)
    return f"File uploaded securely as {filename}", 200

@app.route("/uploads/<path:filename>")
def uploaded_file(filename):
    return send_from_directory(app.config["UPLOAD_FOLDER"], filename)

if __name__ == "__main__":
    app.run(debug=True)
```

```python
from flask import Flask, request, render_template_string
import os

app = Flask(__name__)
UPLOAD_FOLDER = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'uploads')
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

@app.route('/', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            return 'No file part', 400
        file = request.files['file']
        if file.filename == '':
            return 'No selected file', 400
        file.save(os.path.join(UPLOAD_FOLDER, file.filename))
        return 'File uploaded'
    return render_template_string('''
        <form method="post" enctype="multipart/form-data">
            <input type="file" name="file">
            <input type="submit" value="Upload">
        </form>
    ''')

if __name__ == '__main__':
    app.run(debug=True)
```

Done, 1 change Keep Undo

```
PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed>  c:; cd 'c:\Users\dussa\OneDrive\Desktop\blog-app-fixed' ; & 'c:\Users\dussa\AppData\Local\Programs\P
on313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59908' '--' 'c:\Users\dussa\
esktop\Traning\ai.py'
 * Serving Flask app 'ai'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 100-446-805
```



Explanation

Security Audit (File Uploads)

- **Vulnerability:** Trusting the original filename and allowing any file extension.
- **The Risk: Remote Code Execution (RCE)** via uploaded scripts and **Path Traversal** attacks.
- **The Fix:** Use secure_filename() and a strict **Extension Whitelist** (e.g., only .jpg, .png).
- **Rule:** Always rename and validate uploaded files on the server side.

Task 5 – Insecure Data Serialization Check
Task:
Evaluate AI-generated code that uses pickle for saving user data.
Instructions:

- Ask AI to generate a script using pickle.load().
- Explain why untrusted pickle data is dangerous.
- Replace with safer formats like JSON.

Expected Output:
• Secure serialization code using JSON instead of pickle.

AI Prompt Used
"Generate a Python script that uses pickle.dump() to save user profile data and pickle.load() to read it back. Explain why loading untrusted pickle data is a security risk (Remote Code Execution). Then, rewrite the script using the json module to make it secure."

Code:

```
import json
import pickle
from pathlib import Path

def save_profile_pickle(profile: dict, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("wb") as handle:
        pickle.dump(profile, handle)

def load_profile_pickle(path: Path) -> dict:
    with path.open("rb") as handle:
        return pickle.load(handle)

def save_profile_json(profile: dict, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as handle:
        json.dump(profile, handle, indent=2)

def load_profile_json(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as handle:
        return json.load(handle)

def main() -> None:
    profile = {
        "username": "jdoe",
        "email": "jdoe@example.com",
        "preferences": {"theme": "dark", "notifications": True},
        "login_count": 42,
    }

    pickle_path = Path("profiles/user_profile.pkl")
    json_path = Path("profiles/user_profile.json")
```

```
    save_profile_pickle(profile, pickle_path)
    save_profile_json(profile, json_path)

    loaded_pickle = load_profile_pickle(pickle_path)
    loaded_json = load_profile_json(json_path)

    print("Pickle loaded:", loaded_pickle)
    print("JSON loaded:", loaded_json)

if __name__ == "__main__":
    main()
```

<untitled>

```python
import pickle
import json
def save_profile_pickle(profile, filename):
    with open(filename, "wb") as f:
        pickle.dump(profile, f)
def load_profile_pickle(filename):
    with open(filename, "rb") as f:
        return pickle.load(f)
def save_profile_json(profile, filename):
    with open(filename, "w", encoding="utf-8") as f:
        json.dump(profile, f, indent=2)
def load_profile_json(filename):
    with open(filename, "r", encoding="utf-8") as f:
        return json.load(f)
if __name__ == "__main__":
    user_profile = {
        "name": "Alice",
        "email": "alice@example.com",
        "age": 28,
        "preferences": {"theme": "dark", "notifications": True},
    }
    # Save with pickle
    save_profile_pickle(user_profile, "profile.pkl")
    loaded_profile_pickle = load_profile_pickle("profile.pkl")
    print("Loaded from pickle:", loaded_profile_pickle)

    # Save with JSON
    save_profile_json(user_profile, "profile.json")
    loaded_profile_json = load_profile_json("profile.json")
    print("Loaded from json:", loaded_profile_json)

```

Shell

```
>>> %Run -c $EDITOR_CONTENT
 Loaded from pickle: {'name': 'Alice', 'email': 'alice@example.com', 'age': 28, 'preferences': {'theme': 'dark', 'notifications': True}}
 Loaded from json: {'name': 'Alice', 'email': 'alice@example.com', 'age': 28, 'preferences': {'theme': 'dark', 'notifications': True}}
>>>
```

Explanation

Insecure Serialization

- **Vulnerability:** Using the pickle module to save/load user-controlled data.
- **The Risk:** pickle.load() can execute arbitrary system commands, leading to a full system compromise.
- **The Fix:** Replace pickle with **JSON**. JSON is a data-only format and cannot execute logic.
- **Rule:** Avoid pickle for any data that crosses a trust boundary.